

Deep Learning

From Practice to Advanced Architectures

Part 2 | Hands-On Datasets • VGG & ResNet • Transfer Learning • YOLO • RNNs & Transformers

PRACTICE

MNIST • Fashion MNIST • CIFAR-10 •
Kaggle

ARCHITECTURES

VGG • ResNet • Transfer Learning •
YOLO

NEW DOMAINS

RNNs • LSTM • Transformers •
Attention



Today's Agenda

01 Hands-On: MNIST, Fashion MNIST, CIFAR-10

Building & training CNNs on benchmark datasets

02 Advanced Architectures: VGG & ResNet

Deep networks, skip connections, residual learning

03 Transfer Learning

Feature extraction & fine-tuning pre-trained models

04 Object Detection with YOLO

From classification to real-time detection

05 Beyond CNNs: RNNs & Transformers

Sequential data, attention, and the future of DL

06 Kaggle & Next Steps

Competitions, resources, and project ideas

1

Core Datasets Overview

MNIST

28×28 • Grayscale • 60K train

Handwritten digits (0–9)

The "Hello World" of DL

TARGET ACCURACY

99%+

Fashion MNIST

28×28 • Grayscale • 60K train

Clothing items (10 classes)

Harder: shirts vs pullovers

TARGET ACCURACY

93%+

CIFAR-10

32×32 • RGB • 50K train

Real objects (animals, vehicles)

Complex scenes, color images

TARGET ACCURACY

95%+

Progression: MNIST (learn basics) → Fashion MNIST (learn regularization) → CIFAR-10 (learn advanced techniques)

1

Building a CNN for MNIST

1

LOAD

Load & normalize
pixel values $\rightarrow [0, 1]$

2

RESHAPE

Add channel dim
(28, 28) $\rightarrow$ (28, 28, 1)

3

BUILD

Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$
Pool
 $\rightarrow$ Flatten $\rightarrow$ Dense

4

COMPILE

Adam optimizer
Sparse Cat. CE

5

TRAIN

10 epochs, batch 128
Validation split 10%

6

EVALUATE

Test accuracy
 $\approx 99.0\text{--}99.3\%$

Architecture

Conv2D(32, 3 $\times$ 3, ReLU) $\rightarrow$ MaxPool(2 $\times$ 2) $\rightarrow$ Conv2D(64, 3 $\times$ 3, ReLU) $\rightarrow$ MaxPool(2 $\times$ 2)
 $\rightarrow$ Flatten $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(10, Softmax)

Key Details

- Total parameters: $\sim 225\text{K}$ (lightweight, trains in minutes)
- Dropout(0.5) prevents overfitting on the dense layer
- `sparse_categorical_crossentropy` because labels are integers (0–9), not one-hot
- Normalize pixels to $[0, 1]$ for faster, more stable training

1

Fashion MNIST & Data Augmentation

Why Harder Than MNIST?

- High intra-class variation (shirts look very different)
- Low inter-class distinction (shirt $\approx$ pullover $\approx$ coat)
- Best simple CNN: $\sim 91\text{--}93\%$ (vs $99\%+$ for MNIST)
- Requires regularization techniques to perform well

Improved Architecture

- Double conv per block + BatchNormalization
- padding='same' to preserve spatial dims
- Progressive dropout: $25\% \rightarrow 25\% \rightarrow 50\%$
- 256 dense units before softmax output

Data Augmentation — Key Technique

Artificially increase training data by applying random transformations during training. The model sees slightly different versions each epoch, learning invariance to these changes.

Rotation

$\pm 10^\circ$

Handles tilted images

Width/Height Shift

$\pm 10\%$

Handles off-center objects

Zoom

$\pm 10\%$

Handles scale variations

Horizontal Flip

True

Doubles effective data for symmetric objects

1

CIFAR-10: The Real Challenge

What Makes CIFAR-10 Hard?

- 32×32 RGB images — very small for real objects
- Complex backgrounds and cluttered scenes
- Similar classes: cat vs dog, automobile vs truck
- Simple CNN: only ~70–75% accuracy

Deep CNN Pattern

Block 1	32 filters, Dropout 0.2
Block 2	64 filters, Dropout 0.3
Block 3	128 filters, Dropout 0.4
Head	Dense 256, Dropout 0.5

Training Strategies for CIFAR-10

ReduceLROnPlateau

Halves learning rate when validation loss plateaus for 3 epochs

EarlyStopping

Stops training when no improvement; restores best weights

Aggressive Augmentation

Rotation $\pm 15^\circ$, shifts, flips, zoom for better generalization

BatchNormalization

Normalizes layer outputs for faster, more stable training

2

VGG — The Power of Depth & Simplicity

Core Design Philosophy

- Only 3×3 convolution filters everywhere (smallest meaningful size)
- Only 2×2 max pooling to halve dimensions
- Double filters after each pool: 64 → 128 → 256 → 512 → 512
- Two 3×3 convs = same receptive field as one 5×5, fewer params, more non-l.

VGG16 at a Glance

Layers	16 weight layers
Parameters	138 million
Model Size	~528 MB
Top-1 Acc	71.3% (ImageNet)

VGG16 Architecture

Block 1

Conv3-64 ×2

MaxPool

Block 2

Conv3-128 ×2

MaxPool

Block 3

Conv3-256 ×3

MaxPool

Block 4

Conv3-512 ×3

MaxPool

Block 5

Conv3-512 ×3

MaxPool

FC

4096 → 4096 →
1000

Softmax

2

ResNet — Skip Connections Revolution

The Degradation Problem

Before ResNet, deeper networks performed WORSE:

20-layer network: 92% accuracy

56-layer network: 90% accuracy ← worse!

Not overfitting — a fundamental training problem.
Gradients vanish through many layers.

The Solution: Residual Learning

Instead of learning $H(x)$, learn residual $F(x)$:

Output = $F(x) + x$ (skip connection)

- If optimal = identity, just learn $F(x) \approx 0$
- Gradients flow directly through shortcuts
- Can train 100+ layers effectively

ResNet Variants

Model	Params	Top-1 Acc
ResNet-18	11.7M	69.8%
ResNet-34	21.8M	73.3%
ResNet-50	25.6M	76.1%
ResNet-101	44.5M	77.4%
ResNet-152	60.2M	78.3%

VGG16 vs ResNet-50

Parameters	138M vs 25.6M
Model Size	528 MB vs 98 MB
ImageNet Acc	71.3% vs 76.1%
Depth	16 vs 50 layers

3 Transfer Learning

Take a model trained on ImageNet (1.2M images, 1000 classes) and adapt it for your task with far less data.

Feature Extraction

Freeze the entire pre-trained model

- Load pre-trained model (e.g., VGG16, ResNet50)
- Set `base_model.trainable = False`
- Add new Dense layers for your classes
- Train only the new head layers
- Best for: small datasets, similar to ImageNet

Fine-Tuning

Unfreeze top layers, retrain with low LR

- Start from feature extraction model
- Unfreeze last few layers of base model
- Use very small learning rate (1e-5)
- Train to adapt features to your domain
- Best for: medium+ datasets, different domain

Popular Pre-trained Models

Model	Params	Size	Best For
MobileNetV2 3.4M • 14 MB	EfficientNetB0 5.3M • 29 MB	ResNet50 25.6M • 98 MB	VGG16 138M • 528 MB

4

Object Detection with YOLO

Classification vs Detection vs Segmentation

Classification

"This is a cat"

Detection

Cat at (x1,y1,x2,y2)

Segmentation

Every pixel labeled

YOLO = You Only Look Once

Single-pass detection: divides image into grid, each cell predicts boxes + classes simultaneously.
Real-time: 45+ FPS (vs seconds for older methods).

How YOLO Works

1. Grid

Divide image into S×S grid

2. Predict

Each cell predicts B bounding boxes

3. Classify

Assign class probabilities

4. NMS

Remove duplicate detections

5. Output

Final boxes + labels

Key Concepts

- IoU (Intersection over Union) — measures box overlap, 0=none, 1=perfect
- mAP (Mean Average Precision) — standard detection metric, higher = better
- Anchor Boxes — pre-defined box shapes as starting points for predictions
- NMS — removes duplicate detections, keeps highest-confidence box only

Recurrent Neural Networks (RNNs)

CNNs handle spatial data (images). RNNs handle sequential data where order matters.

How RNNs Work

- Process sequences one element at a time
- Maintain hidden state (memory) across steps
- Same weights reused at every time step
- $h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$
- Output depends on current input + past memory

Applications

- Sentiment analysis (tweet → positive/negative)
- Machine translation (English → French)
- Text generation (prompt → story)
- Speech recognition (audio → text)
- Time series forecasting (stocks, weather)

The Vanishing Gradient Problem & Solutions

RNNs struggle with long sequences — gradients shrink to ~ 0 after 20–30 steps, losing long-range memory.

LSTM

3 gates (forget, input, output) + cell state highway. Learns what to remember/forget.

GRU

Simplified LSTM with 2 gates (update, reset). Fewer parameters, often similar performance.

5

Transformers — The Architecture Behind Modern AI

Why Transformers Beat RNNs

- Parallel processing (not sequential → much faster)
- Handles long-range dependencies naturally
- Scales to billions of parameters
- "Attention Is All You Need" (2017)

Self-Attention (Core Idea)

Every token attends to every other token:

$$Q \text{ (Query)} \times K \text{ (Key)}^T / \sqrt{d_k} \rightarrow \text{Softmax} \rightarrow \times V \text{ (Value)}$$

Multi-head: run multiple attention heads in parallel for different relationship types.

Transformer Architecture

Embedding

Text → vectors
+ position info

Multi-Head Attention

Token-to-token
relationships

Feed-Forward

Refine each
token repr.

Encoder

Process input
(BERT)

Decoder

Generate output
(GPT)

Models Built on Transformers

Model	Type	By	Use Case
BERT	Encoder	Google	Text understanding
GPT-4	Decoder	OpenAI	Text generation
T5	Enc-Dec	Google	Translation
ViT	Encoder	Google	Image classification

Kaggle Competitions & Resources

Competitions to Try Today

Digit Recognizer (MNIST)

Target: >99% | kaggle.com/competitions/digit-recognizer

Fashion MNIST

Target: >93% | kaggle.com/competitions/fashion-mnist-competition

CIFAR-10

Target: >95% | kaggle.com/c/cifar-10

Kaggle Strategy

- Start with basic CNN → submit baseline
- Add BatchNorm + Dropout → improve
- Apply data augmentation → generalize
- Try transfer learning → big jump
- Ensemble multiple models → top scores

Learning Resources

- TensorFlow Tutorials — tensorflow.org/tutorials
- PyTorch Tutorials — pytorch.org/tutorials
- Dive into Deep Learning — d2l.ai (free textbook)
- CS231n (Stanford) — YouTube (CNNs)
- CS224n (Stanford) — YouTube (NLP)
- Fast.ai — free practical course

Foundational Papers

- AlexNet (2012) — Started the DL revolution
- VGG (2014) — Depth matters
- ResNet (2015) — Skip connections
- YOLO (2015) — Real-time detection
- Attention Is All You Need (2017)
- ViT (2020) — Transformers for vision

Next Lecture Preview — Part 3

From theory to hands-on implementation and real-world deployment

NLP with Deep Learning

Tokenization, word embeddings, sentiment analysis with LSTM, fine-tuning BERT via Hugging Face

Transfer Learning Lab

Build a custom classifier on your own dataset, compare feature extraction vs fine-tuning

Generative Models: GANs

Generator vs discriminator, build a GAN for MNIST, autoencoders for reconstruction

Custom Object Detection

Annotate your own data, train YOLOv8 on custom objects, real-time webcam detection

Model Deployment & Production

Flask/FastAPI REST APIs, TF Lite for edge devices, MLOps: versioning & monitoring

Before Next Lecture — Homework

- Submit to at least one Kaggle competition (MNIST, Fashion MNIST, or CIFAR-10)
- Experiment with transfer learning — load ResNet50 or VGG16 for CIFAR-10
- Read one foundational paper — the ResNet paper is surprisingly readable
- Explore Hugging Face (huggingface.co) — browse models, try a demo